Computer Organization & Architecture Open-Book Binder

Midterm (UTS) · Sessions 1–7 · CPEN6260010 · Dr. Endra · Linda Null, *Essentials of Computer Organization and Architecture* (2023)

English, with Indonesian terms in *(kurung / brackets)* · Companion to your 2-page cheat sheet

Contents *(Daftar isi)*

Tab	Section	Cheat sheet	Page
G	Glossary: acronyms, register names, and Indonesian question words. **Read this first when a word confuses you.**	all	**p.3**
I	**Problem-type index**: "the question says X → use method Y → go to page Z"	all	**p.5**
C	Calculation toolkit: powers of 2, base conversion by hand, hex arithmetic, the "drop bits / keep bits" trick	T1, T9, T10	**p.7**
1	Introduction: organization vs architecture, Moore's Law, the level hierarchy, von Neumann	T3, C	**p.9**
2	Data representation: binary, hex, bytes, two's complement, characters, endianness	T1, T2, T12	**p.11**
3	CPU basics: datapath, control unit, buses, clock, memory addressing	T3, T9, C	**p.14**
4	MARIE: registers, instruction set, RTN, fetch–decode–execute, assembling and tracing programs	T4, T5, T6	**p.16**
5	Instruction set architectures: formats, operands, postfix, expanding opcodes, addressing modes	T7, T13, T14	**p.20**
6	Performance and pipelining: CPU time, pipeline time, speedup, hazards	T3, T8	**p.24**
7	Memory and cache: hierarchy, locality, mapping, address fields, tracing, replacement, write policies	T9, T10, T11	**p.26**
8	Virtual memory and real-world memory systems	C	**p.29**
X	Traps and errata: mistakes in the slides and common exam traps	all	**p.30**
T	Reference tables: powers of 2, hex/decimal/binary, ASCII, MARIE reference card, formula list	all	**p.31**
Q	**Practice quiz** (new questions, not on the cheat sheet). Do it closed-solutions, open-binder.	all	**p.33**
K	Practice quiz answer key: full worked solutions	all	**p.35**

How to use this binder

1. **Learn in order, 1 → 8.** Each tab starts from zero and builds up. Read the theory, then work every **Example** box with a pencil *before* reading its solution.
2. **Close each tab with its "Check yourself" box.** If you can answer those without looking, you have the tab.
3. **Connect to the cheat sheet.** Every tab header lists its cheat-sheet boxes (T4) and the quiz/practice cards that use it (Q4, P4). After a tab, redo those cards on the cheat sheet. The words on the back of the cheat sheet are summaries of the pages here.
4. **Test yourself with Tab Q**, then mark it with Tab K. Every K answer links back to the page with the method.

5. **In the exam**: classify the question with Tab I, then use the cheat sheet. Show the formula and the substitution, the way the prof's quiz solutions do.

> ✓ Every number in the examples and the answer key was recomputed by computer. The MARIE programs were run in a small MARIE simulator and every cache trace was replayed. Where a slide has a mistake, the corrected fact is used and the slip is listed in Tab X.

Sources: the lecture decks S1 Introduction, S2–S3 MARIE, S4–S5 Instruction Set Architectures, S6–S7 Memory (Dr. Endra, based on Null 2023), the prof's UTS Study Guide, the quiz and the prof's quiz solutions.

Cheat sheet ↔ binder map

Cheat-sheet box	Topic	Learn it on	Box	Topic	Learn it on
T1	Hex ↔ binary, bytes	**p.11**	T8	Pipelining	**p.24**
T2	Two's complement	**p.11**	T9	Address bits	**p.14**
T3	Moore's Law, CPU time	**p.9**, **p.24**	T10	Cache address fields	**p.27**
T4	MARIE format, opcodes	**p.16**	T11	Cache tracing, policies	**p.27**
T5	Fetch–decode–execute	**p.17**	T12	Endianness	**p.12**
T6	Assembling MARIE	**p.18**	T13	Addressing modes	**p.22**
T7	Expanding opcodes	**p.21**	T14	Formats, postfix	**p.20**
C	Concepts for short-answer questions: **p.9** (intro), **p.14** (CPU, bus), **p.19** (control units, CISC/RISC), **p.26** (memory), **p.29** (virtual memory)				

G · Glossary: acronyms, registers, question words

Most marks lost on "easy" questions come from misreading one word. The words that change the method are marked ★.

Acronyms and register names

Term	Means	Page
AC	Accumulator: MARIE's only data register; holds operands and results	p.16
MAR	Memory Address Register: the **address** being read/written	p.16
MBR	Memory Buffer Register: the **data** going to/from memory	p.16
PC	Program Counter: address of the **next** instruction	p.16
IR	Instruction Register: the instruction being executed now	p.16
InREG / OutREG	Input / output registers (8 bits)	p.16
ALU	Arithmetic Logic Unit: does add, subtract, AND, OR, compare	p.14
CU	Control Unit: decodes instructions, sends control signals	p.14
ISA	Instruction Set Architecture: the instructions a CPU understands and their formats	p.20
RTN / RTL	Register Transfer Notation/ Language: "MAR ← PC" style micro-steps	p.16
M[X]	The **contents** of memory address X (X itself is just the address)	p.16
IR[15–12]	Bits 15 down to 12 of IR (the opcode)	p.16
★ MSB / LSB	Most / least significant **bit** or **byte**. Read the context! Q1 means byte, Q2 means bit.	p.11
EA	Effective address: where the operand actually is	p.22
RPN	Reverse Polish notation = postfix (A B +)	p.20
GPR	General-purpose register architecture	p.20
CISC / RISC	Complex / reduced instruction set computer	p.19
IC, CPI	Instruction count; cycles per instruction	p.24
k, n, τ, t_n	Pipeline stages, instructions, cycle time, non-pipelined time per instruction	p.24
SRAM / DRAM	Static RAM (flip-flops, cache) / dynamic RAM (capacitors, main memory)	p.26
LRU / FIFO / LFU	Least recently used / first in first out / least frequently used	p.28
MMU, TLB	Memory management unit; translation look-aside buffer (cache of page-table entries)	p.29
VLSI, IC	Very-large-scale integration; integrated circuit (in history)	p.9
SaaS/ PaaS/IaaS	Software / Platform / Infrastructure as a Service	p.10

Indonesian question words *(kata soal)*

Indonesian	English → what to do
Hitunglah / Tentukan / Carilah	Calculate / Determine / Find → show formula + substitution
Jelaskan	Explain → 2–3 sentences with the key term
Bandingkan / Bedakan	Compare / Distinguish → table of differences
★ Apakah mungkin ... ? Buktikan	Is it possible? Prove → count bit patterns (expanding opcodes)
Ubahlah / Konversikan	Convert (base, notation)
Telusuri / Lacak eksekusi	Trace the execution → table of AC, PC, memory
Tuliskan kode heksadesimal	Write the hex machine code
Tabel simbol	Symbol table (label → address)
alamat / ruang alamat	address / address space
memori utama / sekunder	main / secondary memory
dapat dialamati per byte / per word	byte-addressable / word-addressable
bilangan biner / heksadesimal	binary / hexadecimal number

komplemen dua	two's complement
bit/byte paling signifikan	most significant bit/byte
urutan byte	byte order (endianness)
kode operasi / operan	opcode / operand
siklus jam / frekuensi	clock cycle / frequency
tahap / segmen (pipeline)	stage / segment
★ rasio percepatan	speedup ratio = old time ÷ new time

★ prosesor yang lebih tua / lebih baru	older (slower) / newer (faster) processor
blok / baris (cache)	block / line
pemetaan langsung	direct mapping
asosiatif penuh / asosiatif set	fully associative / set associative
hit / miss, rasio hit	hit / miss, hit rate
kebijakan penggantian / penulisan	replacement / write policy
memori virtual, halaman, bingkai	virtual memory, page, frame

⚠ **TRAP** "Address" vs "contents". In "M[150] = 4155", 150 is where it lives and 4155 is what is stored there. Every MARIE question tests whether you keep these apart.

I · Problem-type index: "I see … → I do … → page"

Look up the **phrase or setup** in the question, not the chapter name. The last column is the cheat-sheet box for the exam.

The question says / shows …	Method	Page	Sheet
Data representation			
a hex value, "binary representation"	each hex digit → 4 bits	**p.11**	T1
"extract the middle bytes / the MSB / the LSB" of a hex value	split into 2-digit pairs, count from the left (MSB) or right (LSB)	**p.11**	T1
"interpreted as two's complement", "positive or negative?"	look at the leftmost bit; value = unsigned - 2^n if negative	**p.11**	T2
"represent -x in n bits"	+x in binary → flip bits → add 1	**p.11**	T2
characters "packed" in a word, ASCII	1 character = 1 byte (2 hex digits)	**p.12**	T1
"stored at address … big/little endian"	pad to full width, write bytes in order (big) or reversed (little)	**p.12**	T12
Performance			
"performance doubles every d years", older/newer processor	factor = 2^(years/d); older × factor, newer ÷ factor	**p.9**	T3
GHz, MHz, "cycle time"	T = 1/f	**p.14**	T3
instruction count, CPI, clock rate, "which is faster?"	CPU time = IC × CPI ÷ f; speedup = old ÷ new	**p.24**	T3
"k-segment pipeline", "speedup ratio", n tasks	$(k + n - 1)\cdot\tau$ vs $n\cdot t_n$	**p.24**	T8
"stall", "hazard", "branch", "forwarding"	hazard types; add s stall cycles	**p.24**	T8
MARIE			
a 4-digit hex instruction, "decode"	first digit = opcode, last 3 = address	**p.16**	T4
"register transfers", "fetch cycle", "RTN/RTL"	MAR←PC · MBR←M[MAR] · IR←MBR · PC←PC+1, then the instruction's RTN	**p.17**	T5
a program with labels, "hex code", "symbol table"	two-pass assembly	**p.18**	T6
"final value of AC", "what does the program compute?"	trace table (AC, PC, memory)	**p.18**	T6
Skipcond, Jump, loops, if/else	Skipcond 000/400/800 skips the next line	**p.18**	T4
"why is MAR 12 bits and AC 16 bits?"	addresses vs data	**p.16**	C
hardwired vs microprogrammed; CISC vs RISC	comparison tables	**p.19**	C
Instruction sets			
"is it possible to encode … instructions with … operands?"	expanding opcodes: Σ count·2^(operand bits) ≤ 2^(instr bits)	**p.21**	T7
"reverse Polish / postfix", "stack machine"	operator after operands; stack evaluation	**p.20**	T14
"write code on a 3-/2-/1-/0-address machine"	templates	**p.20**	T14
immediate / direct / indirect / indexed, "value loaded"	effective-address rules	**p.22**	T13
Memory			
"how many bits to address a A×B memory"	$\log_2$(# addressable units)	**p.14**	T9

"format of the address as seen by the cache", tag/index/offset	offset = $\log_2$ block, index = $\log_2$ lines (or sets)	**p.27**	T10
"to which cache block/set does address X map?"	drop offset bits, keep index bits	**p.26**	T10
a list of block references, "hit or miss"	trace table per line/set	**p.27**	T11
LRU / FIFO / LFU	replacement rules	**p.28**	T11
write-through vs write-back, dirty bit	write policies	**p.28**	T11
hit rate, "effective access time"	EAT = $H \cdot t_{cache} + (1-H) \cdot t_{mem}$	**p.26**	C
SRAM/DRAM, locality, hierarchy goals	concept tables	**p.26**	C
page table, page fault, virtual → physical address	split VA into page # | offset	**p.29**	C
Concepts (Session 1)			
organization vs architecture	"visible to the programmer?"	**p.9**	C
von Neumann, bottleneck, Harvard, parallel	definitions	**p.10**	C
level hierarchy, generations, cloud, Moore's limits	tables	**p.10**	C

C · Calculation toolkit: doing the arithmetic by hand

Cheat sheet: T1 T9 T10 · Used in: Q1, Q2, Q4, Q5, Q8, P1, P2, P8, P9

Almost every CoA calculation is one of five moves: a power of 2, a base conversion, hex addition/subtraction, splitting a bit string, or a "divide by 2^k". Learn these once and the rest of the binder is easy.

1 · Powers of 2 you must know by heart

2^n	value	2^n	value
2^0	1	2^8	256
2^1	2	2^9	512
2^2	4	2^{10}	1,024 = 1K
2^3	8	2^{11}	2,048 = 2K
2^4	16	2^{12}	4,096 = 4K
2^5	32	2^{16}	65,536 = 64K
2^6	64	2^{20}	1,048,576 = 1M
2^7	128	2^{30}	≈ 1.07×10^9 = 1G

Prefix rule: K = 2^{10}, M = 2^{20}, G = 2^{30} (for memory sizes).
So 4M = $2^2 \times 2^{20} = 2^{22}$, 16K = $2^4 \times 2^{10} = 2^{14}$, 8M = 2^{23}, 2M = 2^{21}.
$\log_2$ of a power of 2 is just the exponent: $\log_2(1024) = 10$, $\log_2(16) = 4$.
Multiplying powers: add exponents. $2^4 \times 2^8 = 2^{12}$.
Dividing: subtract. $2^{24} \div 2^4 = 2^{20}$.

⚠ **TRAP** For **speed** (GHz, MHz) and **time** (ns), the prefixes are decimal: 1 GHz = 10^9 Hz, 1 ns = 10^{-9} s. Only memory sizes use powers of 2.

2 · Base conversions by hand

Binary → decimal: add the place values of the 1-bits.
Place values: ... 128 64 32 16 8 4 2 1
$1100\ 1000_2$ = 128 + 64 + 8 = **200**

Decimal → binary: subtract the largest power of 2 that fits, repeat.
200 - 128 = 72 - 64 = 8 - 8 = 0 → bits 128, 64, 8 → **1100 1000**

Hex → decimal: digits × powers of 16 (1, 16, 256, 4096).
0x3C7 = 3·256 + 12·16 + 7 = 768 + 192 + 7 = **967**

Decimal → hex: divide by 16, keep remainders (read upward).
200 ÷ 16 = 12 r 8 → C8. 60 ÷ 16 = 3 r 12 → **3C**

Hex ↔ binary is the one you use most: each hex digit is exactly 4 bits.
0x3C7 → 3 = 0011, C = 1100, 7 = 0111 → **0011 1100 0111**
$1011\ 0110_2$ → 1011 = B, 0110 = 6 → **0xB6**
Group from the **right**; pad the left group with 0s.
1 0110 1101 → 0001 0110 1101 → 0x16D

✓ Learn the 16 rows in Tab T (p.31) until you can write them without thinking. A, C and F (1010, 1100, 1111) come up constantly.

3 · Hex addition and subtraction (MARIE's AC works in hex)

Work digit by digit from the right, like decimal, but carry/borrow at **16**.
Add: 0x2A + 0x3C: A + C = 10 + 12 = 22 = 16 + 6 → write 6, carry 1. 2 + 3 + 1 = 6 → **0x66** (= 42 + 60 = 102 ✓)
Subtract: 0x0035 - 0x0010 = **0x0025** (no borrow needed).
0x0100 - 0x0001: 0 - 1 needs a borrow: 16 - 1 = F, then borrow again → **0x00FF**.

Example (quiz Q4)
AC = 0035, M[155] = 0010, instruction Subt 155. AC ← 0035 - 0010 = **0025** (hex).
In decimal that is 53 - 16 = 37, and 0x25 = 37 ✓.
Give the answer in hex, the way the prof does, and add the decimal if you like.

⚠ **TRAP** When a MARIE value is written as hex (0035), do the arithmetic in hex. 0035 - 0010 is 0025 hex, not 25 decimal.

4 · The "drop bits / keep bits" trick (dividing by 2^k)

For any binary number:
$\lfloor x \div 2^k \rfloor$ = drop the lowest k bits.
$x \bmod 2^k$ = keep only the lowest k bits.
In hex, each hex digit is 4 bits, so ÷16 = drop the last hex digit, mod 16 = keep the last hex digit.
0x5A3C8F ÷ 16 → 0x5A3C8 · 0x5A3C8F mod 16 → 0xF

✔ This is exactly how cache mapping works (**p.26**): the block number is "address ÷ block size" (drop the offset bits), and the cache line is "block mod lines" (keep the index bits). You never need long division.

5 · Splitting a bit string into fields

Example: split 24 bits as Tag 10 | Index 10 | Offset 4 quiz Q8

0x5A3C8F → 0101 1010 0011 1100 1000 1111 (24 bits)
Count from the **right**: offset = last 4 bits **1111**; index = next 10 bits **11 1100 1000**; tag = the 10 bits left **0101 1010 00**.
Index → decimal: 11 1100 1000 = 512 + 256 + 128 + 64 + 8 = **968**. In hex: regroup from the right, 11 1100 1000 = 0x3C8.

✔ Always count field widths from the right (low bits). The left-most field gets whatever is left, and it may not be a multiple of 4.

Check yourself

1. $2^{16} \div 2^3 = ?$ · 2. 0xC8 in decimal? · 3. 0x00FF + 0x0001? · 4. 0xBEEF mod 8? · 5. 13 in 4-bit binary?

Answers: $2^{13} = 8192$ · 200 · 0x0100 · 7 (last 3 bits of F = 1111 are 111) · 1101

1 · Introduction (Session 1, Null Ch. 1)

Cheat sheet: T3 C · Used in: Q3, P3 and short-answer theory questions

Organization vs architecture *(organisasi vs arsitektur)*

Computer architecture = the logical, programmer-visible design: the instruction set, opcodes, data types, number and size of registers, addressing modes. It answers *"How do I design a computer?"*

Computer organization = how that design is physically built: control signals, buses, memory technology, the datapath. It answers *"How does a computer work?"*

▣ **METHOD** Ask: **"Would a programmer writing assembly code notice it?"** Yes → architecture. No (only an engineer building it would) → organization.

Example (the classic textbook case)

"Does the CPU have a multiply instruction?" → programmer sees it → **architecture**.
"Is multiply done by a dedicated multiplier circuit or by repeated addition?" → invisible to the programmer → **organization**.
"How many registers?" → architecture. "Is the cache made of SRAM?" → organization.

The three core components and HW/SW equivalence

Every computer has: (1) a **processor (CPU)** to interpret and execute programs, (2) a **memory** to store both data and programs, (3) an **I/O system** to move data to and from the outside world.

Principle of equivalence of hardware and software: anything done in software can be done in hardware, and anything done in hardware can be done in software. Hardware is almost always **faster**; software is more **flexible** (easy to change). Designers choose based on this speed-vs-flexibility trade-off.

Example: reading a computer ad (slides S1 #8)

"**4.20 GHz**" → CPU clock rate, 4.2×10^9 cycles/s, so one cycle ≈ 0.238 ns (**p.14**).
"**32GB DDR4 SDRAM**" → main memory: 32×2^{30} bytes of dynamic RAM (**p.26**).
"**128KB L1 cache**" → the small fast memory closest to the CPU (**p.26**).

Standards organizations let equipment from different makers work together: **IEEE** (components, signalling, data representation, e.g. floating point) and **ISO** (international). Some are informal trade groups.

Generations and Moore's Law

Generation	Years	Technology
1st	1945–1953	Vacuum tubes (ENIAC)
2nd	1954–1965	Transistors
3rd	1965–1980	Integrated circuits (IC)
4th	1980–	VLSI (very-large-scale integration)

The two driving factors behind all of this: **cost** and **performance** (S1 exercise 1).

Moore's Law: the number of transistors on a chip doubles about every **18–24 months** (≈ 2 years). It is an observation, not a law of physics, and transistor count is not the same as performance.
If performance doubles every **d** years:
Performance factor = 2^(years ÷ d)
Execution time ∝ 1 ÷ performance.

Example: older processor quiz Q3, prof's solution

200 s now; performance doubles every 2 years; processor 4 years **older**.
1. Doublings = 4 ÷ 2 = 2.
2. Factor = 2^2 = 4. The older CPU is 4× **slower**.
3. Time = 200 × 4 = **800 s**

Example: newer processor cheat sheet P3

60 s now; doubling every 1.5 years; processor 6 years **newer**.
1. Doublings = 6 ÷ 1.5 = 4.
2. Factor = 2^4 = 16. Newer → 16× **faster**.
3. Time = 60 ÷ 16 = **3.75 s**

⚠ **TRAP** Older → slower → time gets **bigger** (multiply). Newer → faster → time gets **smaller** (divide). Write "older = slower" next to your working so you don't flip it.

Why Moore's Law can't last forever (S1 exercise 3): transistors cannot shrink below the size of atoms; smaller, denser chips produce more heat and power per area; quantum effects make tiny transistors unreliable; and the cost of each new fabrication plant keeps rising (Rock's Law). Growth has already slowed.

The computer level hierarchy

Level	Name	What lives there
6	User	Running programs (apps)
5	High-level language	C, Java, Python
4	Assembly language	Mnemonics, e.g. MARIE "Load X"
3	System software	Operating system
2	Machine (ISA)	Binary machine instructions
1	Control	Microcode or hardwired control unit
0	Digital logic	Gates, wires, circuits

Each level is a "virtual machine" with its own language. **Abstraction** lets you work at one level without knowing every detail below it. The hierarchy bridges the **semantic gap** between high-level languages and logic gates.

This course lives mostly at levels 1–2 (MARIE's machine code and control) and the memory hardware under them.

Cloud computing, the Internet, IoT

Service	You get	Example
SaaS	A finished application	Gmail
PaaS	A platform (OS, database) to build your own apps on	app-hosting platforms
IaaS	Raw hardware: servers, storage, network	rented virtual servers

Each service matches a level of the hierarchy: SaaS ≈ user level, PaaS ≈ system software, IaaS ≈ machine level.

SCADA networks run infrastructure (power, water). The **Internet of Things (IoT)** is the larger set of connected devices (appliances, security systems). The Internet is robust but **fragile**: an outage or attack can have serious consequences.

The von Neumann model and beyond

Von Neumann architecture (almost every general-purpose computer):
• **Stored-program concept**: instructions and data are stored in the **same** memory.
• Three hardware systems: CPU (with ALU, registers, control unit), main memory, I/O.
• Instructions are executed **sequentially** through the fetch–decode–execute cycle (**p.17**).
• A **single path** between CPU and memory: the **von Neumann bottleneck**, because instructions and data compete for it.

Purpose of the processing unit (S1 exercise 2): it fetches instructions from memory, decodes them, and executes them. The ALU does the arithmetic and logic; the control unit directs the flow of data and instructions.

Non-von Neumann designs break one of those rules. The **Harvard architecture** has **separate** memories/buses for instructions and data, which removes the shared-path bottleneck. **Parallel processing** (many cores or many computers working at once) is the most popular non-von Neumann approach.

Check yourself

1. "MARIE has 16-bit instructions" — architecture or organization? · 2. Name the von Neumann bottleneck. · 3. 120 s now, doubling every 2 years: time on a CPU 6 years newer? · 4. Which level holds the operating system? · 5. Give two reasons Moore's Law must end.

Answers: architecture (visible to the programmer) · the single shared CPU–memory path · $2^3 = 8$ → 15 s · level 3 · atomic size limit, heat/power density, quantum effects, fab cost

2 · Data representation (S1–S3, Null Ch. 2 & 5, Study Guide §4)

Cheat sheet: T1 T2 T12 · Used in: Q1, Q2, P1, P2, P10

Binary and hexadecimal

A computer stores everything as **bits** (0/1). Numbers use positional notation: each position is worth a power of the base.

Unsigned value = Σ bit × 2^position (position 0 is the rightmost)
n bits give 2^n patterns → unsigned range **0 … 2^n - 1**
8 bits: 0…255 · 12 bits: 0…4095 · 16 bits: 0…65,535

Hexadecimal (base 16, digits 0–9, A–F) is shorthand for binary: **1 hex digit = 4 bits**, so 1 byte (8 bits) = 2 hex digits and a 16-bit MARIE word = 4 hex digits.

Hex	Bin	Dec	Hex	Bin	Dec
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

▣ **METHOD** Hex → binary: replace each digit by its 4 bits. Binary → hex: group 4 bits from the right, replace each group. Conversions to/from decimal: Tab C (p.7).

Bytes, MSB and LSB

A 32-bit value has 4 bytes. Written in hex, each byte is one pair of digits:

0x778899AA	77	88	99	AA
role	MSB			LSB

The **most significant byte** (MSB) is the leftmost pair; the **least significant byte** (LSB) is the rightmost pair. The **most significant bit** is the single leftmost bit.

Example: middle bytes — quiz Q1, prof's solution

0x778899AA → drop the highest byte (77) and lowest byte (AA) → middle bytes 88 and 99.
16-bit code: **8899**. Binary: 8 = 1000, 8 = 1000, 9 = 1001, 9 = 1001
0x8899 = 1000 1000 1001 1001

Example: MSB then LSB — cheat sheet P1

0xC0FFEE42 → MSB C0, LSB 42 → **0xC042 = 1100 0000 0100 0010**

⚠ **TRAP** "MSB" can mean most significant **byte** (Q1: "ignoring the most significant byte") or **bit** (Q2: "the MSB, the leftmost bit"). Read which one the question says.

Two's complement: negative integers

In n-bit **two's complement**, the leftmost bit has weight **-2^{n-1}** instead of $+2^{n-1}$.
Range: **-2^{n-1} … 2^{n-1} - 1**
8-bit: -128 … 127 · 16-bit: -32,768 … 32,767
Sign check: MSB (leftmost bit) 0 → non-negative, 1 → negative.
In hex: first digit 0–7 → positive, 8–F → negative.

▣ **METHOD: write -x**
1. Write +x in n bits. 2. Flip every bit. 3. Add 1.

METHOD: decode a negative pattern
Value = (unsigned value) - 2^n.
Or: flip the bits, add 1, read the magnitude, put a minus sign.

Why use it? The same adder circuit works for positive and negative numbers, and there is only one zero.

Example: -10 in 8 bits — Study Guide p.3

+10 = 0000 1010 → flip → 1111 0101 → +1 → **1111 0110** (= 0xF6)
Check: 0xF6 = 246 unsigned; 246 - 256 = -10 ✓

Example: -37 in 8 bits — cheat sheet P2b

+37 = 0010 0101 → 1101 1010 → +1 → **1101 1011 = 0xDB**

Example: decode 0xF3A0 (16-bit) cheat sheet P2a

First digit F ≥ 8 → negative. Unsigned = 15·4096 + 3·256 + 10·16 = 62,368.
Value = 62,368 - 65,536 = **-3168**

Sign extension: to widen a two's-complement number, copy the sign bit to the left. -5 = 1111 1011 (8-bit) → 1111 1111 1111 1011 (16-bit).

Example: decode 0xFFD6 (16-bit)

Unsigned 65,494 → 65,494 - 65,536 = **-42**. (Flip: 0000 0000 0010 1001, +1 → 0010 1010 = 42 ✓)

Overflow: adding two positives that gives a "negative" (or two negatives giving a positive) means the result did not fit. 8-bit: 100 + 50 = 150 > 127 → the bits read as -106.

Characters: the same bits, different meanings

Characters are stored as numeric codes. **ASCII** uses 7 bits, stored in one 8-bit byte: 'A' = 0x41, 'M' = 0x4D, 'E' = 0x45, 'a' = 0x61, '0' = 0x30 (table **p.31**).

A bit pattern has no meaning by itself. 0x4D45 can be the text "ME", the integer 19,781, or a MARIE instruction (4 = Subt, address D45). The interpretation depends on what the program does with it.

Example: packed characters quiz Q2, prof's solution

"ME" stored as 0x4D45 (one character per byte).
Binary: 4 = 0100, D = 1101, 4 = 0100, 5 = 0101 → **0100 1101 0100 0101**
Read as two's complement: the MSB (leftmost bit) is **0** → **positive** (+19,781).

Endianness: byte order in memory *(urutan byte)*

In a byte-addressable memory, a multi-byte value occupies several consecutive addresses.
Endianness is the order its bytes go in.
Big endian: the **most** significant byte at the **lowest** address ("big end first").
Little endian: the **least** significant byte at the **lowest** address.
Only the byte order changes; the bits inside each byte are untouched.

▣ **METHOD**
1. Pad the value to its full width (32-bit = 8 hex digits).
2. Split into bytes (pairs of hex digits).
3. Big: write them left to right from the start address. Little: write them in reverse.
Reading memory back: undo the steps.

Value, start addr		+0	+1	+2	+3
0x12345678	Big	12	34	56	78
	Little	78	56	34	12
0x1234 (32-bit)	Big	00	00	12	34
	Little	34	12	00	00
0xDEADBEEF	Big	DE	AD	BE	EF
	Little	EF	BE	AD	DE
−2 (32-bit) = 0xFFFFFFFE	Big	FF	FF	FF	FE
	Little	FE	FF	FF	FF

Example: reading memory back cheat sheet P10b

Little-endian memory holds 78 56 34 12 at addresses 0–3. The lowest address holds the LSB, so reverse: **0x12345678**

	Big endian	Little endian
Used by	Most UNIX machines, newer RISC (MIPS), network byte order	Intel x86 PCs
Advantages	Reads naturally in a memory dump; sign is in the first byte; strings and integers stored in the same order	Easier multi-byte arithmetic (start from the low byte); a value can be read at a smaller width from the same address

Bi-endian machines can do either: MIPS (S6-7 #19) and ARM. Byte order matters whenever data moves between machines or into files: both sides must agree.

⚠ **TRAP** The S4-5 #16 table says Intel is big endian. That is wrong: Intel is **little endian** (the S4-5 #6 and #15 text slides say so too). See Tab X.

Check yourself

1. 0x3C7 in binary? · 2. Is 0x9ABC positive or negative in 16-bit two's complement? · 3. -1 in 16 bits (hex)? · 4. 8-bit two's complement range? · 5. Store 0x0A0B0C0D little endian at address 100: what is at 101?

Answers: 0011 1100 0111 · negative (9 ≥ 8) · 0xFFFF · -128 … 127 · 0C (order 0D 0C 0B 0A)

3 · CPU basics, buses, clock, memory addressing (Sessions 2–3, Null Ch. 4)

Cheat sheet: T3 T9 C · Used in: P9, P13 and theory questions

Inside the CPU

The CPU **fetches** instructions, **decodes** them and **performs** the operations on the right data. It has two parts:

- **Datapath**: the network of registers, the ALU, and the internal buses that move and process data.
- **Control unit**: the "traffic manager / policeman". It fetches and decodes instructions and sends control signals so the right data is in the right place at the right time.
- **Registers**: the fastest storage, inside the CPU. Some are special-purpose (PC, IR, MAR, MBR), some general-purpose.
- **ALU**: does arithmetic (add, subtract, multiply) and logic (AND, OR, compare). It is part of the CPU and is driven by signals from the control unit (S2-3 exercise 1).

The bus

A **bus** is a shared set of wires connecting subsystems; it moves bits in parallel. A system bus has three kinds of lines:

Lines	Carry
Data	the actual information being moved
Address	the memory or I/O location being accessed
Control	READ / WRITE, timing, which device may use the bus

✓ Link to MARIE: the address lines are fed by MAR, the data lines by MBR. The width of the address bus decides how many locations can be reached: 12 address lines → 2^{12} = 4096 = 4K, which is MARIE's memory size.

The clock

The clock synchronizes every component. One instruction takes a fixed number of clock cycles.
f = 1 / T and **T = 1 / f** (f = clock rate in Hz, T = cycle time in s)
1 GHz = 10^9 Hz ↔ T = 1 ns · 2 GHz ↔ 0.5 ns · 500 MHz ↔ 2 ns

A faster clock does not guarantee a faster program: the cycle must be long enough for signals to cross the circuits, and the number of cycles per instruction also matters (Tab 6).

Example: clock arithmetic
A 2.5 GHz CPU: T = 1 ÷ (2.5 × 10^9) = 0.4 × 10^{-9} s = **0.4 ns**.
An instruction needing 5 cycles takes 5 × 0.4 = **2 ns**.
A 4.20 GHz CPU: T = 1/4.2 ≈ 0.238 ns.

Memory organization and address bits

Memory is a grid of bits: each row (location) has a unique address. A memory written **A × B** has **A locations** each **B bits** wide (e.g. 4M × 16).

- **Byte-addressable**: every byte has its own address (most common today).
- **Word-addressable**: every word has its own address (MARIE).

Address bits = $\log_2$(number of addressable units) (round up if not a power of 2)
Word-addressable: units = number of words (A).
Byte-addressable: units = A × (B ÷ 8).
The number of bits depends on **how many units**, not on their size.

Example: 2M × 32, word-addressable S2-3 exercise 2
Units = 2M = 2 × 2^{20} = 2^{21} → **21 bits**

Example: 4M × 16, byte-addressable S6-7 exercise 2

16 bits = 2 bytes per word. Units = 4M × 2 = 8M = 2^{23} → **23 bits**

Example: MARIE

4K × 16, word-addressable: 4K = 2^{12} → 12 bits. That is why MARIE's MAR and PC are 12 bits wide.

Example: 8M × 32 both ways cheat sheet P9

Word: 8M = 2^{23} → **23**. Byte: 8M × 4 = 32M = 2^{25} → **25**

⚠ **TRAP** In "4M × 16", the 16 is the word width in **bits**. Divide by 8 to get bytes per word before multiplying for a byte-addressable memory.

Check yourself

1. Which bus lines does MAR drive? · 2. Cycle time of a 400 MHz clock? · 3. Address bits for 64K × 16, word- and byte-addressable? · 4. What decides the address width: number of words or word size?

Answers: address lines · 2.5 ns · 16 and 17 (128K) · the number of addressable units

4 · MARIE: a simple computer (Sessions 2–3, Null Ch. 4)

Cheat sheet: T4 T5 T6 · Used in: Q4, Q5, P4, P5

MARIE = *Machine Architecture that is Really Intuitive and Easy*. It is a teaching computer small enough to trace by hand. If you understand MARIE, you understand how every CPU runs a program.

Architecture and registers

- Stored-program, fixed word length, **word-addressable**, accumulator-based.
- **4K words** of memory = 2^{12} → **12-bit addresses** (000 … FFF hex).
- **16-bit** data words and **16-bit** instructions.
- Two's-complement integers.
- Seven registers (table).

Why is MAR 12 bits but AC 16 bits? S2-3 exercise 3

MAR holds an **address**. There are 2^{12} locations, so 12 bits are enough to name any of them. AC holds **data**: a full 16-bit memory word. Addresses and data have different sizes, so their registers do too. (PC also holds an address → 12 bits; MBR and IR hold whole words → 16 bits.)

Reg	Bits	Holds
AC	16	Accumulator: data values and arithmetic results
MAR	12	Memory Address Register: the address to read/write
MBR	16	Memory Buffer Register: data just read from, or about to be written to, memory
PC	12	Program Counter: address of the **next** instruction
IR	16	Instruction Register: the instruction now being executed
InREG	8	Input from the keyboard
OutREG	8	Output to the display

✔ The CPU never touches memory directly. Every read goes memory → MBR, and every write goes MBR → memory, at the address in MAR.

Instruction format and instruction set

bits	15 14 13 12	11 … 0
field	opcode (4 bits)	address (12 bits)
hex	1 digit	3 digits

Written in hex, a MARIE instruction is **opcode digit + 3-digit address**. 4 opcode bits allow 16 instructions; the slides use opcodes 1–9.

Example: decoding Study Guide p.6

130C → opcode 1, address 30C → **Load 30C**
2450 → **Store 450** · 33A1 → **Add 3A1**
4155 → **Subt 155** (quiz Q4) · 320A → **Add 20A** (P4)

Op	Instruction	Meaning
1	Load X	AC ← M[X]
2	Store X	M[X] ← AC
3	Add X	AC ← AC + M[X]
4	Subt X	AC ← AC - M[X]
5	Input	AC ← value from the keyboard
6	Output	display the value in AC
7	Halt	stop the program
8	Skipcond	skip the next instruction if the condition holds
9	Jump X	PC ← X (continue at X)

⚠ **TRAP** X is an **address**; M[X] is the **value stored there**. "Add 107" adds the contents of address 107, not the number 107.

Register Transfer Notation (RTN)

One instruction is really several tiny steps (**micro-operations**). RTN writes each as "destination ← source".

Instr	RTN	ALU?
Load X	MAR ← X · MBR ← M[MAR] · AC ← MBR	no
Store X	MAR ← X · MBR ← AC · M[MAR] ← MBR	no

Add X	MAR ← X · MBR ← M[MAR] · AC ← AC + MBR	yes
Subt X	MAR ← X · MBR ← M[MAR] · AC ← AC - MBR	yes
Input	AC ← InREG	no
Output	OutREG ← AC	no
Jump X	PC ← X	no

▣ **METHOD** For any instruction ask three questions (Study Guide p.5):
1. Where does the **address** go? → MAR.
2. Where does the **data** go? → through MBR, to/from AC.
3. Is the **ALU** needed? → only for Add and Subt.

Load: the data flows X → MAR → M[MAR] → MBR → AC (the slide's flow diagram). Store runs the opposite way: AC → MBR → M[MAR].

The fetch–decode–execute cycle

Stage	RTN	In words
Fetch	MAR ← PC	put the next instruction's address on the bus
	MBR ← M[MAR]	read that instruction from memory
	IR ← MBR	keep it in IR
	PC ← PC + 1	point PC at the following instruction
Decode	opcode = IR[15–12]; MAR ← IR[11–0]	control unit splits opcode and address; operand address goes to MAR
Execute	(the instruction's RTN)	e.g. MBR ← M[MAR] · AC ← AC - MBR

At the start of each cycle the CPU checks for an **interrupt** (an event that changes the normal flow).

Worked example quiz Q4, prof's solution

PC = 150, M[150] = 4155, M[155] = 0010, AC = 0035.
1. Decode 4155: opcode **4 (Subt)**, operand address **155**.
2. Fetch: MAR ← PC (150) · MBR ← M[MAR] (4155) · IR ← MBR (4155) · PC ← PC + 1 (151)
3. Execute Subt 155: MAR ← IR[11–0] (155) · MBR ← M[MAR] (0010) · AC ← AC - MBR
Final AC = 0035 - 0010 = **0025 (hex)**

Worked example cheat sheet P4

PC = 200, M[200] = 320A, M[20A] = 0007, AC = 0012. Decode: Add 20A. Fetch: MAR ← 200, MBR ← 320A, IR ← 320A, PC ← 201. Execute: MAR ← 20A, MBR ← 0007, AC ← 0012 + 0007 = **0019**, PC = **201**

⚠ **TRAP** PC is incremented during **fetch**, so during execute it already points to the next instruction (151, not 150). A Jump simply overwrites it.

Skipcond and Jump: decisions and loops

Skipcond (opcode 8) tests the AC. Bits 11–10 of the instruction pick the test:
8000: skip if AC < 0 · **8400**: skip if AC = 0 · **8800**: skip if AC > 0
"Skip" means PC ← PC + 1, so the next instruction is jumped over.
Jump X (opcode 9): PC ← X.

If/else pattern: compute A - B, Skipcond on the result, put a Jump to the "else" part right after it.

Example: loop that multiplies (Prod = A × Cnt)

100 Loop,	Load Prod	110E
101	Add A	310B
102	Store Prod	210E
103	Load Cnt	110C
104	Subt One	410D
105	Store Cnt	210C
106	Skipcond 400	8400
107	Jump Loop	9100
108	Load Prod	110E
109	Output	6000
10A	Halt	7000
10B A,	Dec 4	0004
10C Cnt,	Dec 3	0003
10D One,	Dec 1	0001
10E Prod,	Dec 0	0000

Each pass adds A to Prod and counts Cnt down. When Cnt hits 0, Skipcond 400 skips the Jump. Output: **12** (= 4 × 3).

Assemblers and the symbol table

An **assembler** translates assembly (mnemonics + labels) into machine code. Labels are names for addresses, so the assembler needs two passes:

▣ **METHOD: assemble a MARIE program**
Pass 1 — symbol table: walk the listing, note every label and its address (label → address).
Pass 2 — machine code: for each line, write opcode digit + the 3-digit address of its operand (looked up in the symbol table).
• No operand (Input, Output, Halt) → address **000**.
• Skipcond keeps its condition: 8000 / 8400 / 8800.
• **DEC n** / **HEX n** lines are data: write the value as 4 hex digits (DEC 60 = 003C, HEX 0000 = 0000).

Worked example

quiz Q5, prof's solution

100 Start, LOAD P · 101 ADD Q · 102 SUBT R · 103 STORE S · 104 OUTPUT · 105 HALT · 106 P, DEC 60 · 107 Q, DEC 40 · 108 R, DEC 15 · 109 S, HEX 0000
Symbol table: Start 100 · P 106 · Q 107 · R 108 · S 109

Instr	Op	Addr	Hex
LOAD P	1	106	1106
ADD Q	3	107	3107
SUBT R	4	108	4108
STORE S	2	109	2109
OUTPUT	6	000	6000
HALT	7	000	7000

(Data words in memory: 003C, 0028, 000F, 0000.)

Tracing a program

Keep a table with one row per executed instruction: **address · instruction · AC after · memory changed**. Follow PC, not the line order, when there are jumps.

Addr	Instr	AC (dec)	Memory
—	initial	0	M[106]=60, M[107]=40, M[108]=15
100	LOAD P	60	
101	ADD Q	60 + 40 = 100	
102	SUBT R	100 - 15 = 85	
103	STORE S	85	M[109] = 85

104	OUTPUT	85	prints 85
105	HALT	85	stop

Quiz Q5 (iii): final AC = **85** (0055 hex).

Example with jumps

S2-3 exercise 4

100 LOAD A · 101 ADD One · 102 JUMP S1 · 103 S2, ADD One · 104 STORE A · 105 HALT · 106 S1, ADD A · 107 JUMP S2 · 108 A, HEX 0023 · 109 One, HEX 0001

Hex: 1108 · 3109 · 9106 · 3109 · 2108 · 7000 · 3108 · 9103 · 0023 · 0001

Trace (hex): Load A → 0023 · Add One → 0024 · Jump 106 · Add A → 0024 + 0023 = 0047 · Jump 103 · Add One → 0048 · Store A → M[108] = 0048 · Halt

Final: **AC = A = 0048**

Example with Skipcond

cheat sheet P5

A = 9, B = 14. Load A (9) · Subt B (-5) · Skipcond 000: AC < 0 → skip the Jump · Load B (14) · Jump Done · Store M → M = **14**. The program stores max(A, B).

Control units, CISC and RISC

	Hardwired control	Microprogrammed control
How	fixed combinational logic gates generate the control signals	a small program (microcode) in control memory generates them
Speed	fast	slower
Changing it	hard to design and modify	flexible: rewrite the microcode

	CISC	RISC
Instructions	many, complex, variable length; one can do several operations	few, simple, fixed length; one operation each
Memory access	many instructions can use memory	load/store only
Examples	Intel x86	MIPS, ARM

Check yourself

1. Decode 2109. · 2. Write the fetch RTN from memory. · 3. Machine code for "Skipcond 800"? · 4. A program has 104 HALT and 105 X, DEC 10. Hex for "Load X" and the data word? · 5. After fetching the instruction at 2A0, what is PC?

Answers: Store 109 · MAR←PC, MBR←M[MAR], IR←MBR, PC←PC+1 · 8800 · 1105 and 000A · 2A1

5 · A closer look at instruction set architectures (Sessions 4–5, Null Ch. 5)

Cheat sheet: T7 T13 T14 T12 · Used in: Q6, P6, P12, P14

Design decisions

An ISA designer balances: how much **memory** programs take, how **complex decoding** is, how **long** instructions are, and how **many** instructions there are. Short instructions take less memory and are fetched faster, but leave fewer bits for opcodes and operands.

- **Instruction length**: fixed (fast to decode, may waste space) or variable (compact, harder to decode).
- **Number of operands** per instruction: 0, 1, 2 or 3.
- **Operand location**: immediate (in the instruction), register, or memory.
- **Addressing modes** (p.22) and **byte order** (p.12).

Internal storage	Operands	Pros / cons
Stack	implicit, on top of a stack (0-address)	good code density; the stack can be a bottleneck
Accumulator	one operand is always AC (1-address) — MARIE	very short instructions; lots of memory traffic
General-purpose registers (GPR)	several registers (2- or 3-address)	registers are fast and easy for compilers; most common today

Load/store architecture: only LOAD and STORE touch memory; everything else works on registers (MIPS, ARM).

0-, 1-, 2- and 3-address instructions

The same expression **Z = X × Y + W × U** on each kind of machine (Null's standard example):

3-address	2-address	1-address (AC)
`MULT R1, X, Y`	`LOAD R1, X`	`LOAD X`
`MULT R2, W, U`	`MULT R1, Y`	`MULT Y`
`ADD Z, R1, R2`	`LOAD R2, W`	`STORE Temp`
	`MULT R2, U`	`LOAD W`
	`ADD R1, R2`	`MULT U`
	`STORE Z, R1`	`ADD Temp`
		`STORE Z`

3-address: OP dest, src1, src2 (dest ← src1 OP src2)
2-address: OP dest, src (dest ← dest OP src)
1-address: OP X (AC ← AC OP M[X])
0-address: PUSH / POP plus OP on the top two stack items
Fewer addresses → shorter instructions but **more** of them.

Example S4-5 exercise 6

A = (B + C) × (D + E), 3-address:
ADD R1, B, C · ADD R2, D, E · MULT A, R1, R2

Postfix (reverse Polish) and stack machines

Infix puts the operator between operands (2 + 3). **Postfix (RPN)** puts it **after** (2 3 +). Postfix needs no parentheses and is exactly how a stack machine works.

▣ **METHOD: infix → postfix**
1. Fully bracket by precedence (× ÷ before + -, left to right).
2. Inside each bracket write: left operand, right operand, operator.
3. Work from the innermost bracket out.

Example S4-5 exercise 2

(2 + 3) × 8 / 10 = (((2 + 3) × 8) / 10)
→ (2 3 +) → (2 3 + 8 ×) → **2 3 + 8 × 10 /**

Example cheat sheet P14

(A + B) × (C - D) → **A B + C D - ×**

▣ **METHOD: evaluate postfix with a stack**
Read left to right. Operand → push it. Operator → pop two, apply (second-popped OP first-popped), push the result.

Read	Stack after
2	2
3	2 3
+	5
8	5 8
×	40
10	40 10
/	4

Stack-machine (0-address) code is the same sequence: PUSH 2 · PUSH 3 · ADD · PUSH 8 · MULT · PUSH 10 · DIV.

Expanding opcodes

With a fixed instruction length, instructions with **more operand bits** have **fewer opcode bits** left. **Expanding opcodes** give a short opcode to instructions with many operands and longer opcodes to those with few. Unused short opcodes act as **escape opcodes**: they say "the opcode continues into the next field".

Prof's method (pattern count)
An instruction with b operand bits uses **2^b bit patterns** (one for every operand value).
Possible ⇔ Σ (count × 2^(operand bits)) ≤ 2^(instruction bits)

Escape method (constructive check)
1. First group: opcode bits = instruction bits - its operand bits → 2^(opcode bits) codes. Use what you need.
2. Leftover codes × 2^(extra opcode bits) = codes for the next group.
3. Repeat. If any group gets fewer codes than it needs → impossible.

Worked example

quiz Q6, prof's solution

16-bit; 5 instr with three 4-bit operands, 30 with two 5-bit, 50 with one 8-bit.
$5 \times 2^4 \times 2^4 \times 2^4 = 5 \times 2^{12} = 20{,}480$
$30 \times 2^5 \times 2^5 = 30 \times 2^{10} = 30{,}720$
$50 \times 2^8 = 12{,}800$
Total 64,000 ≤ 2^{16} = 65,536 → **possible** (1,536 patterns unused)
Escape check: 4-bit opcode → 16, use 5 → 11 left. 11 × 2^2 = 44 ≥ 30 → 14 left. 14 × 2^2 = 56 ≥ 50 ✓

Example

S4-5 exercise 3

11-bit, 4-bit address fields; 5 two-address, 45 one-address, 32 zero-address.
$5 \cdot 2^8 + 45 \cdot 2^4 + 32 = 1280 + 720 + 32 = 2032 \le 2^{11} = 2048$ → **yes**
Escape: 3-bit opcode → 8, use 5 → 3. $3 \cdot 2^4 = 48 \ge 45$ → 3. $3 \cdot 2^4 = 48 \ge 32$ ✓

Example: impossible

cheat sheet P6

12-bit, 4-bit fields; 14 two-address, 32 one-address, 16 zero-address.
$14 \cdot 2^8 + 32 \cdot 2^4 + 16 = 3584 + 512 + 16 = 4112 > 4096$ → **not possible**. Escape: 2 left → 32, all used → 0 for zero-address.

Example: designing the encoding (16-bit, 4-bit fields: 15 three-address, 14 two-address, 31 one-address, 16 zero-address)

Group	Bit patterns	Count
3-address	0000 R1 R2 R3 … 1110 R1 R2 R3	15
2-address	1111 0000 R1 R2 … 1111 1101 R1 R2	14
1-address	1111 1110 0000 R1 … 1111 1111 1110 R1	31
0-address	1111 1111 1111 0000 … 1111 1111 1111 1111	16

Check: 15·4096 + 14·256 + 31·16 + 16 = 65,536 = 2^{16} exactly; every pattern is used. 1111 is the escape opcode for the first field, 1110/1111 for the second, and so on.

Instruction types

Type	Examples
Data movement	LOAD, STORE, MOVE, PUSH, POP
Arithmetic	ADD, SUBTRACT, MULTIPLY, DIVIDE, INCREMENT, DECREMENT (integer and floating point)
Boolean logic / bit manipulation	AND, OR, XOR, NOT, SHIFT, ROTATE
Input/output	INPUT, OUTPUT (memory ↔ devices)
Transfer of control	branches (conditional/unconditional), JUMP, procedure CALL/RETURN — MARIE's Skipcond and Jump
Special purpose	NOP (no-op), flag control, string processing

Addressing modes: where is the operand?

Mode	Effective address (EA)	Operand
Immediate	— (value in the instruction)	the field itself (ADD R1, #5)
Direct	EA = A	M[A] (LOAD R2, 1000)
Register	— (a register)	the register (ADD R3, R4)
Indirect	EA = M[A]	M[M[A]] — two memory reads
Indexed	EA = A + index register	M[A + R]

Arrays: EA = base + index × element size (element 5 of an array of 4-byte ints at 2000 → 2000 + 20 = 2020).

Direct vs indirect (S4-5 exercise 4): direct gives the operand's address in the instruction; indirect gives the address of a location that **holds** the operand's address. LOAD 100 direct → loads M[100]. LOAD 100 indirect → if M[100] = 250, loads M[250].

Example (Null's standard figure)

Instruction "Load 800". Memory: M[800] = 900, M[900] = 1000, M[1000] = 500, M[1600] = 700.
Index register R1 = 800.
Immediate → **800**
Direct → M[800] = **900**
Indirect → M[M[800]] = M[900] = **1000**
Indexed → EA = 800 + 800 = 1600 → **700**

Example cheat sheet P12

LOAD 300, M[300] = 500, M[500] = 700, M[800] = 900, R1 = 500 → 300 · 500 · 700 · 900

Real-world ISAs

	Intel (x86)	MIPS	ARM
Philosophy	CISC	RISC	RISC, load/store
Operands	2-address (up to 3)	3-address	3-operand
Instruction length	variable, 1–15 bytes	fixed 32-bit	fixed 32-bit (16-bit Thumb)

Byte order	**little endian**	big (bi-endian)	bi-endian
Design goal	backward compatibility	simple, fast pipeline	power efficiency; most widely used 32-bit

Check yourself

1. A × (B + C) - D / E to postfix · 2. Evaluate 6 2 3 + × 4 - · 3. 16-bit, 4-bit fields: 14 three-address and 30 two-address. Possible? And with 40 two-address? · 4. LOAD 200 with M[200] = 350, M[350] = 410: direct and indirect values?

Answers: A B C + × D E / - · 26 · 30: yes (14·4096 + 30·256 = 65,024 ≤ 65,536; 2 escapes × 16 = 32 ≥ 30) · 40: no (14·4096 + 40·256 = 67,584 > 65,536) · 350 and 410

6 · Performance and pipelining (S2–S5, Null Ch. 5, Study Guide §7)

Cheat sheet: T3 T8 · Used in: Q7, P7, P13

The CPU performance equation

CPU cycles = IC × CPI
CPU time = IC × CPI × T = (IC × CPI) ÷ f
CPI = (CPU time × f) ÷ IC
Performance ∝ 1 ÷ execution time
Speedup = old time ÷ new time (> 1 means the new one is faster)
IC = instruction count, CPI = average cycles per instruction, f = clock rate, T = 1/f

A faster clock alone does not guarantee a faster program: instruction count, CPI, memory behaviour and the program itself all matter.

Example

cheat sheet P13

4×10^8 instructions. Old: CPI 2.5 at 2 GHz. New: CPI 3 at 2.5 GHz.
Old = $4\times10^8 \times 2.5 \div 2\times10^9$ = **0.5 s**
New = $4\times10^8 \times 3 \div 2.5\times10^9$ = **0.48 s**
Speedup = 0.5 ÷ 0.48 = **1.042**: the new CPU wins despite its worse CPI.

Example: a faster clock that doesn't help

2×10^9 instructions. A: CPI 1.5 at 3 GHz → $3\times10^9 \div 3\times10^9$ = 1.0 s. B: CPI 1.2 at 2.4 GHz → $2.4\times10^9 \div 2.4\times10^9$ = 1.0 s. Same time: B's better CPI cancels A's faster clock.

Instruction pipelining

Pipelining **overlaps** the stages of the fetch–decode–execute cycle, like an assembly line: while instruction 1 is in stage 2, instruction 2 is in stage 1. It increases **throughput** (instructions finished per second); it does **not** make one instruction finish faster.

Cycle	1	2	3	4	5	6	7	8	9
Instr 1	S1	S2	S3	S4	S5	S6			
Instr 2		S1	S2	S3	S4	S5	S6		
Instr 3			S1	S2	S3	S4	S5	S6	
Instr 4				S1	S2	S3	S4	S5	S6

6 stages, 4 instructions: 6 + 4 - 1 = 9 cycles (slide S4-5 #14).
Without pipelining: 6 × 4 = 24 cycles.

k = stages, n = instructions (tasks), τ = pipeline clock cycle, t_n = time for one instruction without pipelining (= k·τ if the stages are equal)
Pipelined time = (k + n - 1) × τ
Non-pipelined time = n × t_n
Speedup S = n·t_n ÷ ((k + n - 1)·τ)
Equal stages: S = n·k ÷ (k + n - 1) → approaches **k** as n grows.
With s stall cycles: (k + n - 1 + s) × τ

✔ Why k + n - 1? The first instruction needs k cycles to go through every stage ("filling the pipe"). After that, one instruction finishes every cycle: n - 1 more.

Worked example

quiz Q7, prof's solution

Given: k = 5, τ = 40 ns, n = 800, t_n = k × τ = 200 ns.
Step 1 Non-pipelined = n × t_n = 800 × 200 = 160,000 ns
Step 2 Pipelined = (k + n - 1) × τ = (5 + 800 - 1) × 40 = 804 × 40 = 32,160 ns
Step 3 S = 160,000 ÷ 32,160 ≈ **4.975** (close to the maximum 5)

Example

S4-5 exercise 5

Non-pipelined task 200 ns; 5 segments of 40 ns; 200 tasks.
200 × 200 = 40,000 ns vs (5 + 199) × 40 = 8,160 ns → S = **4.90**

Example with stalls

cheat sheet P7

k = 4, τ = 25 ns, n = 100, t_n = 100 ns. S = 10,000 ÷ (103 × 25) = 3.883. With 6 stalls: 10,000 ÷ (109 × 25) = 3.670.

Pipeline hazards

Hazard	What happens	Example	Fix

Structural (resource conflict)	two instructions need the same hardware in the same cycle	one instruction fetches while another reads data from a single-ported memory	stall, or add hardware (separate instruction/data caches)
Data (dependency)	an instruction needs a result that isn't ready yet	ADD R1, R2, R3 then SUB R4, **R1**, R5	stall, or forwarding/ bypassing the result
Control (branch)	the next instruction depends on a branch not yet decided	after a conditional branch, which instruction to fetch?	stall, flush, branch prediction

Check yourself

1. 6-stage pipeline, 15 ns cycle, 500 instructions, t_n = 90 ns: speedup? · 2. Maximum possible speedup of that pipeline? · 3. Which hazard does forwarding fix? · 4. 10^9 instructions, CPI 2, 4 GHz: CPU time?

Answers: 45,000 ÷ (505 × 15) = 45,000 ÷ 7,575 = 5.94 · 6 · data hazards · 0.5 s

7 · Memory and cache (Sessions 6–7, Null Ch. 6)

Cheat sheet: T9 T10 T11 C · Used in: Q8, P8, P11 and theory questions

Memory basics and types

- **Latency**: time for one memory access to complete.
- **Bandwidth**: bits/bytes transferred per second.
- **Static** memory keeps its state while power is on; **dynamic** memory must be refreshed constantly.
- **Volatile** memory loses its contents when power is off (RAM); **nonvolatile** keeps them (ROM, flash, disk).
- **RAM**: every cell has an address and can be accessed in any order in the same time (vs sequential-access memory like tape).

	SRAM	DRAM
Stores a bit in	a flip-flop	a capacitor
Refresh	not needed	needed periodically
Speed	faster	slower
Cost per bit	higher	lower
Used for	**cache**	**main memory**

The memory hierarchy and locality

Registers → Cache → Main memory → Secondary storage (disk)

Going down the hierarchy	Trend
Capacity	increases
Cost per bit	decreases
Latency (access time)	increases
Speed	decreases

Goal (S6-7 exercise 1): give the speed of the fastest memory at roughly the cost per bit of the cheapest, by keeping the data the CPU needs right now in the small, fast levels.

Temporal locality: data used recently will probably be used again soon.
e.g. a loop counter, the instructions inside a loop.
Spatial locality: data near a recently used address will probably be used soon.
e.g. walking through an array, sequential instructions.

Caches exploit both: they keep recently used blocks (temporal) and load a whole **block** of neighbouring bytes on each miss (spatial).

Cache hits, misses and access time

The CPU checks the cache first. Found → **hit**. Not found → **miss**: the whole block is copied from main memory into the cache, then used.

Hit rate H = hits ÷ total accesses · miss rate = 1 - H
Effective access time EAT = H × t_{cache} + (1 - H) × t_{memory}

Example
Cache 10 ns, main memory 200 ns, hit rate 99%.
EAT = 0.99 × 10 + 0.01 × 200 = 9.9 + 2 = **11.9 ns** — nearly cache speed with a 99% hit rate.

The three mapping schemes

Scheme	Where can a memory block go?	Address fields	Pros / cons
Direct mapped	exactly **one** line: line = block mod (# lines)	Tag | Index | Offset	simple, cheap; conflict misses when two hot blocks share a line; no replacement choice
Fully associative	**any** line	Tag | Offset (no index)	most flexible; must search every line in parallel → expensive
N-way set associative	any of the N lines of **one set**: set = block mod (# sets)	Tag | Set | Offset	compromise: good hit rate, moderate cost

Address fields: Tag | Index (Set) | Offset

Address bits = $\log_2$(memory size in addressable units)
Offset = $\log_2$(block size) — which byte inside the block
Index = $\log_2$(# cache lines) — which line (direct mapped)
Set = $\log_2$(# sets), # sets = lines ÷ N — which set (N-way)
Tag = address bits - index (or set) - offset — which memory block is in the line
Fully associative: Tag = address bits - offset

main-memory blocks = memory size ÷ block size
Block number = ⌊address ÷ block size⌋ (drop the offset bits)
Line = block mod lines · Set = block mod sets (keep the low bits)
Tag value = ⌊block ÷ lines⌋ (the remaining high bits)

Worked example

quiz Q8, prof's solution

2^{24} bytes, byte-addressable, direct-mapped, 1024 blocks of 16 bytes.
1. Memory blocks = $2^{24} \div 2^4$ = **2^{20}**
2. Format: address = $\log_2(2^{24})$ = 24 bits. Offset = $\log_2 16$ = 4. Index = $\log_2 1024$ = 10. Tag = 24 - 10 - 4 = 10. → **Tag 10 | Index 10 | Offset 4**
3. Map 0x5A3C8F
Step 1: binary 0101 1010 0011 1100 1000 1111
Step 2: split from the right (10 | 10 | 4): offset **1111**, index **11 1100 1000**, tag **0101 1010 00**
Step 3: index → decimal 512 + 256 + 128 + 64 + 8 = **968**
Hex shortcut: block = drop the last hex digit = 0x5A3C8; line = low 10 bits = 0x3C8 = 968; tag = 0x5A3C8 ÷ 1024 = 0x168 = 360.

Same memory, other mappings

Fully associative: no index → Tag 20 | Offset 4. 0x5A3C8F → tag 0x5A3C8.
4-way set associative: 1024 ÷ 4 = 256 sets → set 8 bits → Tag 12 | Set 8 | Offset 4. 0x5A3C8F → 0x5A3 | 0xC8 | 0xF → **set 200**.

Example

cheat sheet P8

2^{16} bytes, 64-line cache, 8-byte blocks. Blocks = 2^{13}. Direct: offset 3, index 6, tag 7. 0xBEEF = 1011111 | 011101 | 111 → line 29, tag 95. 2-way: 32 sets → 8 | 5 | 3 → 10111110 | 11101 | 111 → set 29, tag 190.

⚠ **TRAP** For a set-associative cache the middle field is $\log_2$(**sets**), not $\log_2$(lines). Fewer index bits means more tag bits.

Tracing a reference string

▣ **METHOD** For each reference: (1) find its line (direct) or set (set-assoc.), (2) hit if that block is already there, otherwise miss and load it (evicting according to the policy), (3) write the cache contents after each step.

All three examples use the S6-7 exercise references **0, 1, 13, 2, 8, 0, 14, 1**.

Direct mapped, 8 blocks

S6-7 exercise 3

Ref	Line (mod 8)	H/M	Note
0	0	M	load 0
1	1	M	load 1
13	5	M	load 13
2	2	M	load 2
8	0	M	evicts 0
0	0	M	evicts 8
14	6	M	load 14
1	1	H	still there

1 hit, 7 misses. Blocks 0 and 8 fight over line 0 (a conflict miss).

4-way set associative, 8 blocks

S6-7 exercise 5

8 ÷ 4 = 2 sets; set = block mod 2. Even blocks → set 0, odd → set 1.

Ref	Set	H/M	Set contents after
0	0	M	{0}
1	1	M	{1}
13	1	M	{1, 13}
2	0	M	{0, 2}
8	0	M	{0, 2, 8}
0	0	H	{0, 2, 8}
14	0	M	{0, 2, 8, 14}
1	1	H	{1, 13}

2 hits. No set ever fills, so no replacement is needed.

Fully associative, 4 blocks: LRU and FIFO

S6-7 exercise 4

Ref	H/M	LRU: evict → contents	FIFO: evict → contents
0, 1, 13, 2	M M M M	fill empty lines → {0, 1, 13, 2}	fill → {0, 1, 13, 2}
8	M	0 (least recently used) → {8, 1, 13, 2}	0 (first in) → {8, 1, 13, 2}
0	M	1 → {8, 0, 13, 2}	1 → {8, 0, 13, 2}
14	M	13 → {8, 0, 14, 2}	13 → {8, 0, 14, 2}
1	M	2 → {8, 0, 14, 1}	2 → {8, 0, 14, 1}

Both: **8 misses, 0 hits**. They agree because no block is hit while it is in the cache, so the least recently used block is always also the oldest.

Replacement policies

Needed only when a miss happens and the line/set is full. Direct-mapped caches have **no choice**: the mapping fixes the line.

Policy	Evicts
FIFO	the block that was **loaded** earliest. Hits change nothing.
LRU	the block **used** least recently. Every hit refreshes the block.
LFU	the block with the fewest uses

Random	a random block

Example where LRU and FIFO differ

cheat sheet P11b

Fully associative, 2 blocks, refs 1, 2, 1, 3, 1, 2.
LRU: 1 M · 2 M · 1 H (1 is now most recent) · 3 M evicts **2** · 1 H · 2 M → **2 hits**
FIFO: 1 M · 2 M · 1 H (order unchanged: 1 is still oldest) · 3 M evicts **1** · 1 M · 2 M → **1 hit**

Write policies

	Write-through	Write-back
On a write	update cache **and** main memory every time	update the cache only; set the block's **dirty bit**
Memory updated	immediately	when the dirty block is replaced
Consistency	memory always current	memory temporarily out of date

Bus traffic	high	low

Example

A loop writes to variable X 100 times while X stays cached.
Write-through: 100 memory writes.
Write-back: 0 memory writes during the loop; 1 write when X's block is finally evicted (its dirty bit is set).

Check yourself

1. 2^{20}-byte memory, direct-mapped, 256 lines of 32 bytes: fields? · 2. Same cache 4-way: fields? · 3. Direct-mapped, 4 lines, refs 0, 4, 8, 0: hits? · 4. Which policy uses a dirty bit? · 5. Loop over an array: which locality?

Answers: Tag 7 | Index 8 | Offset 5 · 64 sets → Tag 9 | Set 6 | Offset 5 · 0 (all map to line 0 and keep evicting each other) · write-back · spatial (and temporal for the loop code)

8 · Virtual memory and real-world memory systems (Sessions 6–7, Null Ch. 6)

Cheat sheet: C · Used in: theory questions (not in the quiz so far)

Why virtual memory

Virtual memory uses the hard disk as an extension of main memory. Each program gets a large **virtual address space**, so it can be bigger than the physical RAM, and programmers no longer worry about physical memory limits.

The most common implementation is **paging**: virtual memory is cut into fixed-size **pages**, physical memory into **frames** of the same size.

Term	Meaning
Virtual address	the logical address the program uses
Physical address	the real location in RAM
Page table	a table **in main memory** mapping virtual page numbers to physical frame numbers (with a valid bit)
Page fault	the page is not in main memory; it must be copied from disk
MMU	hardware that does the translation
TLB	a small cache of recent page-table entries, so most translations skip the page table

Address translation

Virtual address = **page number | offset**
Offset bits = $\log_2$(page size)
Page-number bits = virtual address bits - offset bits
Physical address = **frame number | same offset**

▣ **METHOD**
1. Split the virtual address. 2. Look the page number up (TLB first, then page table). 3. Valid → replace the page number by the frame number, keep the offset. 4. Not valid → **page fault**: the OS loads the page from disk into a free frame (replacing one if needed), updates the table, and the instruction restarts.

Example
16-bit virtual addresses, 4 KB (2^{12}) pages → offset 12 bits, page number 4 bits.
VA 0x3A7F → page **3**, offset **A7F**.
Page table: page 3 → frame 5 (valid).
PA = frame 5 | A7F = **0x5A7F**
If page 3 were marked invalid → page fault.

✔ Same idea as cache fields: the low bits (offset) pass through unchanged; only the high bits are translated.

Real-world memory management

Intel Core i9: three cache levels. L1 is split into an instruction cache and a data cache, each with its own TLB. L1 is 8-way, L2 16-way, L3 11-way set associative.

MIPS: an on-chip MMU maps virtual to physical addresses; caches are initialized by boot code; the TLB is managed by software; supports both big- and little-endian.

Check yourself
1. 1 KB pages and 16-bit virtual addresses: how many pages? · 2. Where is the page table stored? · 3. What is a TLB a cache of? · 4. VA 0x3A7F, page 3 → frame 9: physical address?
Answers: 2^6 = 64 (offset 10 bits) · main memory · page-table entries · 0x9A7F

X · Traps and errata

Mistakes and inconsistencies in the course materials (do not copy these)

Where	Material says	Correct
S4-5 #16 table	Intel: "Big-endian (historically little-endian)"	Intel x86 is **little endian**. The S4-5 #6 and #15 text slides agree. **p.12**
S4-5 #16 table	MIPS and ARM: "Big-endian"	Both are **bi-endian** (S6-7 #19 says so for MIPS). If asked, say "big endian (bi-endian capable)".
S6-7 #10 cache diagram	labels "Dasta", "Vesta", "Sipeed"	Garbled image text. Read it as: CPU ↔ cache (faster) ↔ main memory/RAM (slower) over the system bus.
S1 #12 vs Study Guide p.1	"18–24 months" vs "about every two years"	Same law, different wording. In a calculation, use the period the question gives.
S4-5 #8	"Instruction Formats" slide has only the escape-opcode idea	The counting method the prof uses in Q6 is on **p.21**.
S6-7 #14 and #16	Write-policy slide appears twice	Harmless duplicate. Content on **p.28**.

Exam traps

- **MSB = bit or byte?** Q1 means byte, Q2 means bit. Read the sentence.
- **Older vs newer** (Moore): older → multiply the time, newer → divide.
- **Address vs contents**: X is an address, M[X] is what's stored there.
- **Hex arithmetic in MARIE**: 0035 - 0010 = 0025 *hex*, which is 37 decimal.
- **DEC vs HEX directives**: "DEC 60" is stored as 003C; "HEX 60" is 0060.
- **No-operand instructions** get address 000: Output = 6000, Halt = 7000.
- **Skipcond codes**: 000 → AC < 0, 400 → AC = 0, 800 → AC > 0. It skips the **next** line only.
- **PC during execute** already points to the next instruction.
- **Fetch has 4 transfers**: don't forget PC ← PC + 1, and don't skip the MBR.
- **Execute starts with MAR ← IR[11–0]** (the prof's form), not MAR ← X, once the instruction is in IR.
- **Pipeline formula** is $(k + n - 1)\cdot\tau$, not $k\cdot n\cdot\tau$. The non-pipelined time uses $t_n = k\cdot\tau$.
- **Expanding opcodes**: multiply each count by 2^(**total** operand bits) (three 4-bit operands = 2^{12}).
- **4M × 16 byte-addressable**: × 2 bytes per word → 8M → 23 bits.
- **Cache fields are split from the right**; the tag is whatever is left.
- **Set-associative index** = $\log_2$(**sets**), sets = lines ÷ N.
- **Direct mapped has no replacement policy**: the line is fixed.
- **FIFO ignores hits**; LRU refreshes on every hit.
- **Write-back** uses the dirty bit; write-through doesn't need one.
- **Two's complement sign**: a hex first digit 8–F means negative (16-bit).
- **Endianness**: pad 0x1234 to 00001234 before storing it as 32-bit.
- **K/M/G** are powers of 2 for memory, powers of 10 for Hz and seconds.

T · Reference tables

T.1 Powers of 2

n	2^n	n	2^n
0	1	16	65,536 (64K)
1	2	17	131,072 (128K)
2	4	18	262,144 (256K)
3	8	19	524,288 (512K)
4	16	20	1,048,576 (1M)
5	32	21	2M
6	64	22	4M
7	128	23	8M
8	256	24	16M
9	512	25	32M
10	1,024 (1K)	26	64M
11	2,048 (2K)	27	128M
12	4,096 (4K)	28	256M
13	8,192 (8K)	29	512M
14	16,384 (16K)	30	1G
15	32,768 (32K)	32	4G (4,294,967,296)

T.2 Hex / binary / decimal

Hex	Binary	Dec	×16	×256	×4096
0	0000	0	0	0	0
1	0001	1	16	256	4096
2	0010	2	32	512	8192
3	0011	3	48	768	12288
4	0100	4	64	1024	16384
5	0101	5	80	1280	20480
6	0110	6	96	1536	24576
7	0111	7	112	1792	28672
8	1000	8	128	2048	32768
9	1001	9	144	2304	36864
A	1010	10	160	2560	40960
B	1011	11	176	2816	45056
C	1100	12	192	3072	49152
D	1101	13	208	3328	53248
E	1110	14	224	3584	57344
F	1111	15	240	3840	61440

Use the columns for positional value: 0x3C7 = 768 (3 in ×256) + 192 (C in ×16) + 7 = 967.

T.3 ASCII (printable characters, hex)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2_	spc	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Row = first hex digit, column = second. 'M' = row 4, column D = 0x4D. Lower case = upper case + 0x20.

T.4 MARIE reference card

Hex	Instr	RTN (after fetch & decode)
1X	Load X	MAR ← X · MBR ← M[MAR] · AC ← MBR
2X	Store X	MAR ← X · MBR ← AC · M[MAR] ← MBR
3X	Add X	MAR ← X · MBR ← M[MAR] · AC ← AC + MBR
4X	Subt X	MAR ← X · MBR ← M[MAR] · AC ← AC - MBR
5000	Input	AC ← InREG
6000	Output	OutREG ← AC
7000	Halt	stop
8000/8400/8800	Skipcond	if AC < 0 / = 0 / > 0 then PC ← PC + 1
9X	Jump X	PC ← X

Format: opcode IR[15–12] | address IR[11–0]
Memory: 4K × 16, word-addressable, 12-bit addresses
Fetch: MAR ← PC · MBR ← M[MAR] · IR ← MBR · PC ← PC + 1
Decode: MAR ← IR[11–0], opcode → control unit
Registers: AC 16 · MAR 12 · MBR 16 · PC 12 · IR 16 · InREG 8 · OutREG 8

T.5 Formula list (with cheat-sheet box)

Formula	Box	Formula	Box
patterns with n bits = 2^n; unsigned 0 … 2^n-1	T1	pipelined = (k + n - 1)·τ (+ s stalls)	T8
two's complement -2^{n-1} … 2^{n-1}-1; neg value = unsigned - 2^n	T2	$S = n \cdot t_n \div ((k + n - 1) \cdot \tau) \rightarrow k$	T8
factor = 2^(years ÷ d)	T3	address bits = $\log_2$(# units)	T9
f = 1/T	T3	offset = $\log_2$ block, index = $\log_2$ lines	T10
CPU time = IC × CPI ÷ f	T3	sets = lines ÷ N; set bits = $\log_2$ sets	T10
speedup = old ÷ new	T3, T8	block = ⌊addr ÷ block⌋; line = block mod lines	T10
MARIE word = opcode + 3-hex-digit address	T4	hit rate = hits ÷ accesses; $EAT = H \cdot t_c + (1-H) \cdot t_m$	T11, C
Σ count·2^(operand bits) ≤ 2^(instr bits)	T7	EA: direct A, indirect M[A], indexed A + R	T13

Q · Practice quiz: Computer Organization & Architecture

Rules: open binder, **do not look at Tab K until you finish**. Suggested time 120 minutes for all 19 questions, or about 60 minutes for any 10. Topics are mixed on purpose, so part of the task is choosing the method (Tab I). These questions are **new**: none of them are on the cheat sheet. Show the formula, the substitution and the answer, as in the prof's quiz solutions.

1. (a) Convert 0x3C7 to binary and to decimal. (b) Convert 200 to 8-bit binary and to hex. (c) Compute 0x2A + 0x3C in hex. [5]

2. (a) The 16-bit pattern 0xFFD6 is read as an unsigned integer and as a two's-complement integer. Give both values. (b) Write -100 in 8-bit two's complement (binary and hex). (c) What is the range of a 12-bit two's-complement integer? [6]

3. (a) The string "HI" is packed into one 16-bit word ('H' = 0x48, 'I' = 0x49). Write the word in hex and binary. Read as two's complement, is it positive or negative? (b) For the 32-bit value 0x12AB34CD, give the most significant byte, the least significant byte, and the 16-bit code formed by the two middle bytes. [5]

4. (a) The 32-bit integer -2 is stored at address 0x100. Show the bytes at 0x100–0x103 on a big-endian and on a little-endian machine. (b) A big-endian machine holds 00 00 01 2C at addresses 0x200–0x203. What decimal value is stored there? [5]

5. A program runs in 10 s on today's processor. Performance doubles every 18 months. How long does it take on a processor (a) 3 years older, (b) 4.5 years newer? [4]

6. A memory is organized as 64K × 16. How many address bits are needed if it is (a) word-addressable, (b) byte-addressable? (c) Explain why MARIE's PC is 12 bits wide. [4]

7. A MARIE computer has PC = 0A0, M[0A0] = 20B4, M[0B4] = FFFF and AC = 0123 (all hex). (a) Decode the instruction. (b) Show the fetch-cycle register transfers. (c) Show the execute-cycle transfers and give the final AC, M[0B4] and PC. [7]

8. Consider the MARIE program: [8]

100		LOAD X	107	Done,	OUTPUT
101		SUBT Y	108		HALT
102		SKIPCOND 400	109	X,	DEC 7
103		JUMP Diff	10A	Y,	DEC 7
104		LOAD One	10B	One,	DEC 1
105		JUMP Done	10C	Zero,	DEC 0
106	Diff,	LOAD Zero			

(a) Build the symbol table. (b) Give the hex code for addresses 100–10C. (c) Trace the program and give the output. (d) What is the output if Y is DEC 5 instead? What does the program do?

9. A computer has 16-bit instructions and 4-bit address fields. Is it possible to encode, using expanding opcodes, 13 three-address, 40 two-address, 50 one-address and 16 zero-address instructions? Prove it by counting bit patterns, then confirm with the escape-opcode method. [6]

10. (a) Convert A × (B + C) - D / E to postfix. (b) Evaluate the postfix expression 6 2 3 + × 4 -, showing the stack. (c) Write stack-machine (0-address) code for Z = (A + B) × C. [5]

11. The instruction is LOAD 200. Memory holds M[200] = 350, M[350] = 410, M[600] = 725, and the index register R1 = 400. What value is loaded with immediate, direct, indirect and indexed addressing? [4]

12. A non-pipelined processor takes 90 ns per instruction. A 6-stage pipeline with a 15 ns clock runs the same 500 instructions. (a) Speedup? (b) Maximum possible speedup? (c) Speedup if hazards add 20 stall cycles? (d) Name the hazard caused by "ADD R1, R2, R3" followed by "SUB R4, R1, R5" and one fix. [6]

13. A program executes 2×10^9 instructions. CPU A has CPI 1.5 at 3 GHz; CPU B has CPI 1.2 at 2.4 GHz. Compute both CPU times and the speedup of A over B. What does this show about clock rate? [4]

14. A byte-addressable memory has 2^{20} bytes. The cache has 256 blocks of 32 bytes. (a) How many blocks of main memory are there? (b) Direct mapped: give the address format and the cache block of address 0xABCDE. (c) 4-way set associative: give the address format and the set of 0xABCDE. [7]

15. A cache has 4 blocks. The block references are 0, 4, 8, 0, 4, 1, 5, 1. Show hit/miss for each and the number of hits if the cache is (a) direct mapped, (b) 2-way set associative with LRU, (c) fully associative with LRU. Explain the difference. [6]

16. Cache access time is 5 ns and main memory access time is 80 ns. (a) Effective access time at a 95% hit rate? (b) What hit rate gives an effective access time of 6 ns? [4]

17. (a) A program updates the same variable 50 times while it stays in the cache. How many main-memory writes happen under write-through and under write-back? (b) What is the dirty bit for? (c) A loop sums the elements of an array. Which kinds of locality does it show? [5]

18. (a) Classify as architecture or organization: the number of registers; whether the cache uses SRAM; the instruction format; the bus clock speed. (b) What is the von Neumann bottleneck, and how does the Harvard architecture avoid it? (c) Compare hardwired and microprogrammed control. [5]

19. A system has 16-bit virtual addresses and 1 KB pages. (a) How many bits are the page number and offset? (b) Virtual address 0x1C3A is on a page mapped to frame 2. What is the physical address? (c) What happens if the page-table entry is marked invalid? [4]

— End — Total 100 points. Now mark your work with Tab K.

K · Answer key: worked solutions

The page links point to the method. Answers are highlighted.

1. Conversions → p.7, p.7

(a) 3 = 0011, C = 1100, 7 = 0111 → **0011 1100 0111** ; 3·256 + 12·16 + 7 = **967**
(b) 200 - 128 = 72 - 64 = 8 - 8 = 0 → **1100 1000** = **0xC8**
(c) A + C = 22 = 16 + 6 → 6 carry 1; 2 + 3 + 1 = 6 → **0x66** (42 + 60 = 102 ✓)

2. Two's complement → p.11

(a) Unsigned 15·4096 + 15·256 + 13·16 + 6 = **65,494** . First digit F → negative: 65,494 - 65,536 = **-42**
(b) +100 = 0110 0100 → flip 1001 1011 → +1 → **1001 1100 = 0x9C** (check: 156 - 256 = -100 ✓)
(c) -2^{11} … $2^{11} - 1$ = **-2048 … 2047**

3. Characters and bytes → p.12, p.11

(a) **0x4849 = 0100 1000 0100 1001** . Leftmost bit 0 → **positive** (+18,505)
(b) Bytes 12 | AB | 34 | CD → MSB **0x12** , LSB **0xCD** , middle bytes **0xAB34**

4. Endianness → p.12

(a) -2 in 32 bits = 0xFFFFFFFE. Big: **FF FF FF FE** . Little: **FE FF FF FF**
(b) Big endian reads left to right: 0x0000012C = 1·256 + 2·16 + 12 = **300**

5. Moore's Law → p.9

18 months = 1.5 years. (a) 3 ÷ 1.5 = 2 doublings → 4× slower → 10 × 4 = **40 s** . (b) 4.5 ÷ 1.5 = 3 → 8× faster → 10 ÷ 8 = **1.25 s**

6. Address bits → p.14, p.16

(a) 64K = 2^{16} words → **16 bits** . (b) 64K × 2 bytes = 128K = 2^{17} → **17 bits** . (c) PC holds an address. MARIE has 4K = 2^{12} words, so 12 bits are enough to address every word.

7. MARIE fetch–decode–execute → p.17

(a) 20B4 → opcode 2 = **Store 0B4**
(b) MAR ← PC (0A0) · MBR ← M[MAR] (20B4) · IR ← MBR (20B4) · PC ← PC + 1 (0A1)
(c) MAR ← IR[11–0] (0B4) · MBR ← AC (0123) · M[MAR] ← MBR. Final: **AC = 0123, M[0B4] = 0123, PC = 0A1** (Store doesn't change AC; the old FFFF is overwritten.)

8. Assembling and tracing → p.18, p.18

(a) Symbol table: **Diff 106 · Done 107 · X 109 · Y 10A · One 10B · Zero 10C**
(b) **1109 · 410A · 8400 · 9106 · 110B · 9107 · 110C · 6000 · 7000 · 0007 · 0007 · 0001 · 0000**
(c) Load X (7) · Subt Y (0) · Skipcond 400: AC = 0 → skip 103 · Load One (1) · Jump Done · Output **1** · Halt
(d) Y = 5: AC = 2 ≠ 0 → no skip → Jump Diff · Load Zero · Output **0** . The program outputs 1 if X = Y and 0 otherwise.

9. Expanding opcodes → p.21

Count: 13·2^{12} + 40·2^{8} + 50·2^{4} + 16·2^{0} = 53,248 + 10,240 + 800 + 16 = 64,304 ≤ 65,536 → **possible**
Escape: 16 - 13 = 3 left → 3·16 = 48 ≥ 40 → 8 left → 8·16 = 128 ≥ 50 → 78 left → 78·16 = 1,248 ≥ 16 ✓

10. Postfix and stack code → p.20

(a) ((A × (B + C)) - (D / E)) → **A B C + × D E / -**
(b) push 6 · push 2 · push 3 · + → 6 5 · × → 30 · push 4 · - → **26**
(c) **PUSH A · PUSH B · ADD · PUSH C · MULT · POP Z**

11. Addressing modes → p.22

Immediate **200** · direct M[200] = **350** · indirect M[M[200]] = M[350] = **410** · indexed EA = 200 + 400 = 600 → **725**

12. Pipelining → p.24, p.24

Non-pipelined = 500 × 90 = 45,000 ns. Pipelined = (6 + 500 - 1) × 15 = 7,575 ns.
(a) S = 45,000 ÷ 7,575 = **5.94** (b) k = **6** (c) (505 + 20) × 15 = 7,875 → S = **5.71**
(d) **Data hazard** (SUB needs R1 before ADD has written it). Fix: forwarding/bypassing, or stall.

13. CPU time → p.24

A: $2\times10^9 \times 1.5 \div 3\times10^9$ = **1.0 s**. B: $2\times10^9 \times 1.2 \div 2.4\times10^9$ = **1.0 s**. Speedup = **1**. A's faster clock is cancelled by its higher CPI: clock rate alone does not decide performance.

14. Cache address fields → p.27

(a) $2^{20} \div 2^5$ = **2^{15} = 32,768 blocks**
(b) 20-bit address: offset $\log_2 32 = 5$, index $\log_2 256 = 8$, tag 20 - 8 - 5 = 7 → **Tag 7 | Index 8 | Offset 5**.
0xABCDE = 1010 1011 1100 1101 1110 → 1010101 | 11100110 | 11110 → **block 230** (0xE6), tag 85, offset 30
(c) 256 ÷ 4 = 64 sets → set 6 bits, tag 9 → **Tag 9 | Set 6 | Offset 5**. 101010111 | 100110 | 11110 → **set 38**, tag 343

15. Cache tracing → p.27, p.28

(a) Direct: lines (mod 4) 0, 0, 0, 0, 0, 1, 1, 1 → M M M M M M M M → **0 hits** (0, 4, 8 keep evicting each other in line 0; 1 and 5 do the same in line 1)
(b) 2-way: 2 sets (mod 2). Set 0 gets 0, 4, 8, 0, 4: M M M (evicts 0) M (evicts 4) M (evicts 8). Set 1 gets 1, 5, 1: M M **H** → **1 hit**
(c) Fully associative: 0 M · 4 M · 8 M · 0 **H** · 4 **H** · 1 M (cache full: 0, 4, 8, 1) · 5 M evicts LRU block 8 · 1 **H** → **3 hits**
More flexibility in placement → fewer conflict misses.

16. Effective access time → p.26

(a) 0.95 × 5 + 0.05 × 80 = 4.75 + 4 = **8.75 ns**
(b) 6 = 5H + 80(1 - H) = 80 - 75H → H = 74/75 = **0.9867 (98.7%)**

17. Write policies and locality → p.28, p.26

(a) Write-through **50** writes; write-back **0** while cached, 1 when the block is evicted. (b) It marks a block modified in the cache, so it is written back to memory on replacement. (c) **Spatial** (neighbouring elements) and **temporal** (the sum variable and loop instructions are reused).

18. Concepts → p.9, p.10, p.19

(a) Registers: **architecture**. SRAM cache: **organization**. Instruction format: **architecture**. Bus clock speed: **organization**.
(b) Instructions and data share one memory and one CPU–memory path, so they compete for it. Harvard uses separate memories/paths for instructions and data.
(c) Hardwired: logic gates, fast, hard to change. Microprogrammed: microcode, flexible, slower.

19. Virtual memory → p.29

(a) Offset $\log_2 1024$ = **10 bits**, page number 16 - 10 = **6 bits**
(b) 0x1C3A = 000111 | 0000111010 → page 7, offset 0x03A → frame 2 | 0x03A = **0x083A**
(c) **Page fault**: the OS loads the page from disk into a frame, updates the page table, and the access is retried.

Self-score

If you missed ...	Do this
conversions, two's complement, endianness (1–4)	Redo Tab C and Tab 2, then cheat-sheet Q1, Q2, P1, P2, P10
MARIE (7–8)	Re-trace the examples on **p.17** and **p.18** by hand, then Q4, Q5, P4, P5
expanding opcodes, postfix, modes (9–11)	Tab 5 worked examples, then Q6, P6, P12, P14
pipelines, CPU time (12–13)	Tab 6, then Q7, P7, P13
cache (14–17)	Tab 7 trace tables, then Q8, P8, P11

theory (18–19)	Re-read the comparison tables; cheat-sheet box C